SQL PROJECT

ONLINE BOOKSTORE DATA ANALYSIS

A PostgreSQL portfolio project using relational tables, joins, aggregation, filtering and subqueries.

PROJECT REPORT • SQL • DATA ANALYSIS

Project Overview

This report documents the question set, the SQL work submitted, the source CSV datasets, and a concise set of data findings. The project uses the provided files and PostgreSQL-style SQL.

Dataset files: Books.csv • Customers.csv • Orders.csv

1. Project Overview

The goal of this project is to explore an online bookstore database and answer practical business questions about its books, customers, orders, revenue, genre performance and stock levels. The work starts with importing three CSV files into related tables, then uses SQL to retrieve and summarize information.

500	500	500	7
Books records	Customers records	Orders records	Genres

Tools and techniques

• PostgreSQL — database and SQL query execution

• CSV import — loading Books, Customers and Orders

• Filtering with WHERE and date ranges

• Sorting with ORDER BY and limiting results with LIMIT

• Aggregations using SUM, AVG and COUNT

• Relational analysis using JOIN / LEFT JOIN, GROUP BY and HAVING

• Subquery-based selection for top-priced Fantasy books

Project workflow

1. Create the three tables with primary and foreign keys.
2. Import the CSV files into their matching tables.
3. Run basic retrieval and filtering queries.
4. Use joins and aggregate functions for the advanced questions.
5. Review summary findings and identify query improvements.

Scope note: this report summarizes the provided data snapshot. Findings are descriptive and do not imply business performance beyond these CSV records.

2. Database Design

The schema links each order to one customer and one book. The matching ID columns allow related records to be combined for analysis.

Table	Primary key	Important columns	Relationship
Books	Book_ID	Title, Author, Genre, Published_Year, Price, Stock	Referenced by Orders.Book_ID
Customers	Customer_ID	Name, Email, Phone, City, Country	Referenced by Orders.Customer_ID
Orders	Order_ID	Customer_ID, Book_ID, Order_Date, Quantity, Total_Amount	Foreign keys to Customers and Books

Relationship diagram

Books PK: Book_ID Title • Genre • Price • Stock	**Orders** PK: Order_ID FK: Book_ID FK: Customer_ID Date • Quantity • Amount	**Customers** PK: Customer_ID Name • City • Country
One book can appear in many orders	Each order row connects a book and a customer	One customer can place many orders

Data model note: the SQL script defines Book_ID and Customer_ID as SERIAL primary keys and uses matching integer foreign-key columns in Orders. When importing CSV files with existing IDs, preserve the supplied IDs and check the import mapping.

3. Snapshot Findings

The following metrics were calculated directly from the three uploaded CSV files. Revenue is the sum of Orders.Total_Amount; units ordered is the sum of Orders.Quantity.

$75,628.66	2,697	25,056
Recorded order revenue	Units ordered	Listed stock units

Metric	Result
Genre with the most ordered units	Mystery — 504 units
Average price of Fantasy books	$25.98
Highest listed book price	Proactive system-worthy orchestration — $49.98
Lowest listed stock	Networked systemic implementation — 0 units
Order date range	09 Dec 2022 to 07 Dec 2024
Most valuable customer by recorded order amount	Kim Turner — $1,398.90
Most ordered book by quantity	Realigned multi-tasking installation — 28 units

Interpretation: these are dataset-level results, not audited financial statements. “Recorded order revenue” uses the Total_Amount field exactly as provided and may not equal revenue calculated from Price × Quantity if the data contains discounts or other adjustments.

4. Basic SQL Questions

The original question sheet contains 11 basic questions. Below are readable PostgreSQL solutions aligned to those questions.

1. Retrieve all books in the Fiction genre

```
SELECT *
FROM Books
WHERE Genre = 'Fiction';
```

2. Find books published after 1950

```
SELECT *
FROM Books
WHERE Published_Year > 1950;
```

3. List all customers from Canada

```
SELECT *
FROM Customers
WHERE Country = 'Canada';
```

4. Show orders placed in November 2023

```
SELECT *
FROM Orders
WHERE Order_Date >= DATE '2023-11-01'
  AND Order_Date <  DATE '2023-12-01';
```

5. Retrieve the total stock of books available

```
SELECT SUM(Stock) AS total_stock
FROM Books;
```

6. Find the most expensive book

```
SELECT *
FROM Books
ORDER BY Price DESC
LIMIT 1;
```

7. Show order rows with quantity greater than 1

```
SELECT *
FROM Orders
WHERE Quantity > 1;
```

8. Retrieve orders where total amount exceeds $20

```
SELECT *
FROM Orders
WHERE Total_Amount > 20;
```

9. List unique genres in Books

```
SELECT DISTINCT Genre
FROM Books
ORDER BY Genre;
```

10. Find the book with the lowest stock

```
SELECT *
FROM Books
ORDER BY Stock ASC
LIMIT 1;
```

11. Calculate total recorded revenue

```
SELECT SUM(Total_Amount) AS total_revenue
FROM Orders;
```

5. Advanced SQL Questions

These queries use joins and aggregations to answer the advanced question set. Some are written more precisely than the submitted draft so that the output directly matches the wording of each question.

1. Total number of books sold for each genre

```
SELECT b.Genre, SUM(o.Quantity) AS total_books_sold
FROM Orders o
JOIN Books b ON b.Book_ID = o.Book_ID
GROUP BY b.Genre
ORDER BY total_books_sold DESC;
```

2. Average price of Fantasy books

```
SELECT AVG(Price) AS average_price
FROM Books
WHERE Genre = 'Fantasy';
```

3. Customers who placed at least 2 orders

```
SELECT c.Customer_ID, c.Name,
       COUNT(*) AS order_count
FROM Orders o
JOIN Customers c ON c.Customer_ID = o.Customer_ID
GROUP BY c.Customer_ID, c.Name
HAVING COUNT(*) >= 2
ORDER BY order_count DESC;
```

4. Most frequently ordered book (by units)

```
SELECT b.Book_ID, b.Title,
       SUM(o.Quantity) AS total_order_quantity
FROM Orders o
JOIN Books b ON b.Book_ID = o.Book_ID
GROUP BY b.Book_ID, b.Title
ORDER BY total_order_quantity DESC
LIMIT 1;
```

5. Top 3 most expensive Fantasy books

```
SELECT *
FROM Books
WHERE Genre = 'Fantasy'
ORDER BY Price DESC
LIMIT 3;
```

6. Total quantity sold by each author

```
SELECT b.Author, SUM(o.Quantity) AS total_books_sold
FROM Orders o
JOIN Books b ON b.Book_ID = o.Book_ID
GROUP BY b.Author
ORDER BY total_books_sold DESC;
```

7. Cities of customers with an order over $30

```
SELECT DISTINCT c.City
FROM Orders o
JOIN Customers c ON c.Customer_ID = o.Customer_ID
WHERE o.Total_Amount > 30
ORDER BY c.City;
```

8. Customer who spent the most across orders

```
SELECT c.Customer_ID, c.Name,
       SUM(o.Total_Amount) AS total_spent
FROM Orders o
JOIN Customers c ON c.Customer_ID = o.Customer_ID
GROUP BY c.Customer_ID, c.Name
ORDER BY total_spent DESC
LIMIT 1;
```

9. Stock remaining after fulfilling all orders

```
SELECT b.Book_ID, b.Title, b.Stock AS listed_stock,
       COALESCE(SUM(o.Quantity), 0) AS ordered_quantity,
       b.Stock - COALESCE(SUM(o.Quantity), 0) AS remaining_quantity
FROM Books b
LEFT JOIN Orders o ON o.Book_ID = b.Book_ID
GROUP BY b.Book_ID, b.Title, b.Stock
ORDER BY b.Book_ID;
```

6. Query Review & Improvements

The submitted SQL file demonstrates the main concepts needed for this project. The following refinements help make results match the questions more closely and improve reliability.

Genre sales aggregation

The draft selected Genre and Quantity without grouping. SUM(Quantity) with GROUP BY Genre returns one total per genre.

Unique genre list

Use DISTINCT so each genre appears once.

Cities for customers who spent over $30

Selecting City and Total_Amount can return repeated cities. The revised query returns distinct cities.

Remaining stock

The query subtracts all ordered quantity from listed stock. This assumes the Stock column is starting stock; if it already reflects sales, the calculation would double-count.

Top 3 Fantasy books

ORDER BY Price DESC LIMIT 3 is a direct and readable solution. It returns three rows; ties at the third price may be excluded.

November date filter

Using a half-open date range includes all dates in November and is robust if the date column later changes to a timestamp.

Customers ordering more than one quantity

The basic question was interpreted as order rows where Quantity > 1, matching the submitted query. If it means customers, join Orders to Customers and select distinct customer records.

Example: query to return customer names for order rows with quantity greater than 1

```
SELECT DISTINCT c.Customer_ID, c.Name
FROM Orders o
JOIN Customers c ON c.Customer_ID = o.Customer_ID
WHERE o.Quantity > 1;
```

Import and execution checklist

• Create/use the intended PostgreSQL database before running the table DDL.

• Create Books and Customers before Orders so foreign-key references can be created.

• Import CSV columns into the matching tables and verify column order / data types.

• Check date parsing for Order_Date and confirm IDs were imported correctly.

• Run SELECT COUNT(*) on each table after import and compare with the source CSV row counts.

• For repeatable execution, avoid dropping tables after data has been imported unless you plan to reload the CSVs.

7. Setup SQL Reference

This PostgreSQL DDL follows the structure of the submitted script. Run it only when creating a fresh schema; DROP TABLE removes existing table data.

```
DROP TABLE IF EXISTS Orders;
DROP TABLE IF EXISTS Customers;
DROP TABLE IF EXISTS Books;

CREATE TABLE Books (
    Book_ID SERIAL PRIMARY KEY,
    Title VARCHAR(100),
    Author VARCHAR(100),
    Genre VARCHAR(50),
    Published_Year INT,
    Price NUMERIC(10, 2),
    Stock INT
);

CREATE TABLE Customers (
    Customer_ID SERIAL PRIMARY KEY,
    Name VARCHAR(100),
    Email VARCHAR(100),
    Phone VARCHAR(15),
    City VARCHAR(50),
    Country VARCHAR(150)
);

CREATE TABLE Orders (
    Order_ID SERIAL PRIMARY KEY,
    Customer_ID INT REFERENCES Customers(Customer_ID),
    Book_ID INT REFERENCES Books(Book_ID),
    Order_Date DATE,
    Quantity INT,
    Total_Amount NUMERIC(10, 2)
);
```

CSV files used

Books.csv — 500 records, 7 columns: Book_ID, Title, Author, Genre, Published_Year, Price, Stock.
Customers.csv — 500 records, 6 columns: Customer_ID, Name, Email, Phone, City, Country.
Orders.csv — 500 records, 6 columns: Order_ID, Customer_ID, Book_ID, Order_Date, Quantity, Total_Amount.

Portfolio summary

This Online Bookstore SQL project demonstrates relational data modeling, CSV data loading, data filtering, date conditions, aggregate functions, joins, grouping, HAVING, sorting, subqueries and basic data-quality review. The accompanying query set can be reused to practice or demonstrate PostgreSQL skills in a data analyst portfolio.